

Project Documentation: Educational Phishing Simulator

1. Abstract

This project is a web application developed using **Python** and the **Flask** framework to create a secure simulation of a phishing attack. The project's purpose is purely **educational**: to understand the technical mechanics of phishing attacks by building a functional Proof-of-Concept (PoC). It mimics a multi-step login process similar to a Google sign-in page. The application captures user-inputted data (email and password), stores it in a local database, and displays it on a private dashboard. The entire system operates within a secure local environment (**localhost**) to prevent any potential misuse.

2. Key Features

- **Multi-Step Simulation:** The application emulates a two-step login process (one page for email, a second for the password) to increase realism.
 - **Input Validation:** It includes a feature to validate that the text entered in the email field matches a standard email format.
 - **Data Persistence:** All captured data (username, password, and timestamp) is permanently saved in a local **SQLite** database (**database.db**).
 - **Results Dashboard:** A private web page (**/results**) is available to display all collected data in a clean, organized table.
 - **Responsive User Interface (UI):** The front-end pages are designed to be responsive, adapting to both desktop and mobile screen sizes.
 - **Secure Environment:** The project is configured to run exclusively on the local network, preventing access from the public internet and ensuring it remains an educational tool.
-

3. Technology Stack

- **Backend:**
 - **Language:** Python
 - **Framework:** Flask
 - **Database:** SQLite
 - **Frontend:**
 - **Languages:** HTML, CSS3, JavaScript
 - **Template Engine:** Jinja (included with Flask)
 - **Tools:**
 - **Code Editor:** Visual Studio Code
-

4. Workflow

1. When a user visits the main page, the server renders the `login_email.html` template.
2. The user enters their email. The server validates the format, saves the email in a temporary session, and then renders the `login_password.html` template.
3. The user enters their password.
4. The server receives the password, retrieves the email from the session, and saves both pieces of data together in the `database.db` file.
5. Finally, the user is redirected to the official Google website to complete the simulation.
6. The administrator can visit `http://127.0.0.1:5000/results` at any time to view all the data that has been collected.